

AFRILANG Stdlib

std/point

Français. Bibliothèque AFRILANG 'std/point' (niveau simple, catégorie utilities). Elle expose 2 fonction(s) : distance2d, midpointX.

'import "std/point"' · 'use point'

- 'distance2d(x1 number, y1 number, x2 number, y2 number) → number'

- 'midpointX(x1 number, x2 number) → number' **English.** AFRILANG

library 'std/point' (tier simple, category utilities). Exports 2 function(s): distance2d, midpointX.

'import "std/point"' · 'use point'

- 'distance2d(x1 number, y1 number, x2 number, y2 number) → number'

- 'midpointX(x1 number, x2 number) → number'

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/point' (niveau simple, catégorie utilities). Elle expose 2 fonction(s) : distance2d, midpointX.

'import "std/point"' · 'use point'

```
- `distance2d(x1 number, y1 number, x2 number, y2 number) → number`
- `midpointX(x1 number, x2 number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/point"
use point
```

Niveau : simple · Catégorie : Utilitaires
Helpers simples et fonctions ponctuelles.

3. Référence API

- distance2d(x1 number, y1 number, x2 number, y2 number) → number•
- midpointX(x1 number, x2 number) → number

4. Exemples d'utilisation

```
import "std/point"
use point
```

```
create result = distance2d(/* args */)
say result
```

```
create result = midpointX(/* args */)
say result
```

5. Bonnes pratiques

- Préférez 'import "std/point"' en tête de fichier.
- Lancez 'afri-lang check' avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir /stdlib/ pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module point

```
function distance2d(x1 number, y1 number, x2 number, y2 number) returns number
end
```

```
function midpointX(x1 number, x2 number) returns number
end
end
```

English

1. Overview

AFRILANG library 'std/point' (tier simple, category utilities). Exports 2 function(s): distance2d, midpointX.

'import "std/point"' · 'use point'

```
- `distance2d(x1 number, y1 number, x2 number, y2 number) → number`
- `midpointX(x1 number, x2 number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/point"
use point
```

Tier: simple · Category: Utilities
Simple helpers and single-purpose functions.

3. API reference

- distance2d(x1 number, y1 number, x2 number, y2 number) → number•
- midpointX(x1 number, x2 number) → number

4. Usage examples

```
import "std/point"
use point
```

```
create result = distance2d(/* args */)
say result
```

```
create result = midpointX(/* args */)
say result
```

5. Best practices

- Prefer `import "std/point"` at the top of your file.
- Run `afri-lang check` before shipping.
- Combine with related stdlib modules from the same category.
- See /stdlib/ for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module point

```
function distance2d(x1 number, y1 number, x2 number, y2 number) returns number
end
```

```
function midpointX(x1 number, x2 number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/point"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/point/>

Source (extrait)

```
module point

function distance2d(x1 number, y1 number, x2 number, y2 number) returns number
end

function midpointX(x1 number, x2 number) returns number
end
end
```